

# CareerGuid AI - Step-by-Step Implementation Plan

## Senior Full-Stack Engineering and Project Management Roadmap

CareerGuid AI / SkillSight

June 28, 2026

## CareerGuid AI - Step-by-Step Implementation Plan

**Role: Senior Full-Stack Engineer + Project Manager**

This plan follows the final PRD versions:

Version 0.1 - Prototype

Version 1.0 - MVP

Version 1.1 - ATS Resume Builder

Version 1.2 - RAG + GenAI Roadmap

Version 1.3 - Scalability + Security

Version 1.4 - Progress Tracking + Weekly Reminders

Version 2.0 - Advanced Career Platform

---

## 1. Recommended Build Order

Do not start directly with AI/RAG. First build foundation, authentication, database, onboarding, and dashboard. Then add AI features.

Correct build sequence:

Project Setup

v

Supabase Database + Auth

v

Frontend Layout + Routing

v

Authentication Flow

v

Resume-first Onboarding

v

Resume Upload + Profile Auto-fill

v

Dashboard

v

Skill Gap Analysis

v

Basic Roadmap

v

ATS Resume Builder

v

RAG Roadmap

v

Progress Tracking  
v  
Weekly Reminder Emails  
v  
Payment / Upgrade  
v  
Admin Panel  
v  
Testing  
v  
Deployment  
v  
Final Polish

---

## 2. Phase 0 - Product and Engineering Setup

### Goal

Prepare project structure, documentation, tools, and development workflow.

### What to Build

Create a monorepo structure:

```
careerguid-ai/  
|  
|---- frontend/  
|   `---- React + TypeScript + Vite  
|  
|---- backend/  
|   `---- Node.js + Express + TypeScript  
|  
|---- ai-service/  
|   `---- Python FastAPI  
|  
|---- worker/  
|   `---- BullMQ background workers  
|  
|---- docs/  
|   |---- PRD.pdf  
|   |---- TRD.pdf  
|   |---- WebFlow.pdf  
|   |---- UIUX.pdf  
|   `---- BackendSchema.pdf  
|  
|---- docker-compose.yml  
|---- README.md  
`---- .env.example
```

### Setup Steps

1. Create GitHub repository.
2. Create `main`, `dev`, and feature branches.
3. Setup frontend with Vite React TypeScript.
4. Setup backend with Node.js, Express, TypeScript.
5. Setup AI service with Python FastAPI.

6. Setup basic Docker Compose.
7. Add `.env.example`.
8. Add README with setup instructions.
9. Add project management board: Backlog, Todo, In Progress, Testing, Done.

## Deliverables

- GitHub repo created
  - Project folder structure ready
  - Frontend running locally
  - Backend running locally
  - AI service running locally
  - Basic Docker Compose ready
  - README added
- 

## 3. Phase 1 - Supabase Setup and Database Foundation

### Goal

Set up Supabase as the main backend database, authentication provider, storage, and vector database.

### What to Build

Use Supabase for:

Authentication  
PostgreSQL database  
Storage  
Row Level Security  
pgvector for RAG

### Database Setup Steps

1. Create Supabase project.
2. Enable required extensions:

```
create extension if not exists "uuid-ossf";  
create extension if not exists "pgcrypto";  
create extension if not exists "vector";
```

3. Create enum types.
4. Create core tables:

```
profiles  
profile_field_sources  
resumes  
job_roles  
skills  
skill_aliases  
role_skills  
skill_analyses  
skill_analysis_items  
roadmaps  
roadmap_weeks  
roadmap_tasks  
generated_resumes  
notifications  
email_logs
```

ai\_jobs  
plans  
subscriptions  
usage\_counters

5. Enable Row Level Security.
6. Add RLS policies.
7. Create Supabase Storage buckets:

resumes  
generated-resumes  
template-previews  
knowledge-base-files

8. Add storage ownership policies.
9. Add seed data for job roles and skills.

## Seed Data

Add initial job roles:

Backend Developer  
Frontend Developer  
Full Stack Developer  
Data Analyst  
AI/ML Engineer  
Cloud Engineer  
DevOps Engineer  
Java Developer  
MERN Stack Developer

Add initial skills:

JavaScript  
TypeScript  
React.js  
Node.js  
Express.js  
PostgreSQL  
Supabase  
Redis  
Docker  
REST API  
JWT  
System Design  
Python  
FastAPI  
RAG  
AI/ML

## Deliverables

- Supabase project ready
- Database schema created
- RLS policies enabled
- Storage buckets created
- Job roles and skills seeded
- Supabase keys added to environment variables

## 4. Phase 2 - Backend Base Setup

### Goal

Create backend foundation with routes, middleware, Supabase connection, Redis, validation, and error handling.

### Backend Stack

Node.js  
Express.js  
TypeScript  
Supabase JS SDK  
Redis  
BullMQ  
Zod  
Winston/Pino  
Swagger

### What to Build

Backend folder structure:

```
backend/src/
|
|---- config/
|   |---- supabase.ts
|   |---- redis.ts
|   `---- env.ts
|
|---- middlewares/
|   |---- auth.middleware.ts
|   |---- admin.middleware.ts
|   |---- error.middleware.ts
|   |---- rateLimit.middleware.ts
|   `---- validate.middleware.ts
|
|---- modules/
|   |---- auth/
|   |---- onboarding/
|   |---- resume/
|   |---- skill-gap/
|   |---- roadmap/
|   |---- resume-builder/
|   |---- reminders/
|   |---- billing/
|   `---- admin/
|
|---- queues/
|---- workers/
|---- utils/
|---- routes/
`---- server.ts
```

### Backend Setup Steps

1. Setup Express server.
2. Add global error handler.
3. Add request validation using Zod.

4. Add Supabase client.
5. Add Redis connection.
6. Add auth middleware to verify Supabase JWT.
7. Add admin middleware.
8. Add basic rate limiting middleware.
9. Add health check APIs:

```
GET /api/health
GET /api/health/db
GET /api/health/redis
GET /api/health/ai-service
```

## Deliverables

- Backend server ready
  - Supabase connection working
  - Redis connection working
  - Auth middleware ready
  - Error handling ready
  - Health check APIs working
- 

## 5. Phase 3 - Frontend Base Setup

### Goal

Create app shell, routing, layouts, design system, and reusable components.

### Frontend Stack

```
React.js
TypeScript
Vite
React Router
React Hook Form
Zod
Axios
Zustand / Context API
Plain CSS / CSS Modules
Lucide Icons
```

### Frontend Folder Structure

```
frontend/src/
|
|---- pages/
|   |---- public/
|   |---- auth/
|   |---- onboarding/
|   |---- dashboard/
|   |---- skill-gap/
|   |---- roadmap/
|   |---- resume-builder/
|   |---- billing/
|   |---- settings/
|   `---- admin/
|
```

```
|---- components/  
|   |---- layout/  
|   |---- forms/  
|   |---- common/  
|   |---- roadmap/  
|   |---- resume/  
|   `---- billing/  
|  
|---- services/  
|---- hooks/  
|---- store/  
|---- utils/  
`---- routes/
```

## What to Build First

1. Public layout
2. Auth layout
3. Dashboard layout
4. Protected route component
5. Admin protected route component
6. Toast component
7. Modal component
8. Button component
9. Input component
10. Card component
11. Empty state component
12. Error state component
13. Loading skeleton component

## Routes to Setup

```
/  
/features  
/pricing  
/login  
/signup  
/verify-email  
/forgot-password  
/reset-password  
/onboarding/upload-resume  
/onboarding/review-profile  
/onboarding/success  
/dashboard  
/skill-gap  
/roadmap  
/roadmap/:roadmapId  
/resume-builder  
/profile  
/billing  
/settings  
/admin
```

## Deliverables

- Frontend app shell ready
- Routing ready

- Public layout ready
  - Dashboard layout ready
  - Reusable UI components ready
  - Protected routes working
- 

## 6. Phase 4 - Authentication Flow

### Goal

Build complete signup, login, email verification, forgot password, and session handling.

### What to Build

Pages:

```
/signup  
/login  
/verify-email  
/forgot-password  
/reset-password
```

### Auth Flow

```
Signup  
v  
Supabase creates user  
v  
Email verification / OTP  
v  
Profile row created  
v  
Login  
v  
Check onboarding status  
v  
Redirect user
```

### Redirect Rules

```
If not logged in -> /login  
If email not verified -> /verify-email  
If onboarding not completed -> /onboarding/upload-resume  
If onboarding completed -> /dashboard  
If admin -> /admin
```

### Backend APIs

```
POST /api/auth/register  
POST /api/auth/login  
POST /api/auth/logout  
POST /api/auth/verify-email  
POST /api/auth/forgot-password  
POST /api/auth/reset-password  
GET /api/auth/me
```



## Rate Limit Rules

Successful login capacity: minimum 100 users/minute system-wide

Failed login attempts: 5 failed attempts/minute per IP + email

OTP request: 3 requests/10 minutes per email

Forgot password: 3 requests/15 minutes per email

## Deliverables

- Signup working
  - Login working
  - Email verification working
  - Forgot password working
  - Session handling working
  - Protected routes working
  - Auth redirects working
- 

## 7. Phase 5 - Version 0.1 Prototype

### Goal

Create a basic clickable prototype before full backend integration.

### What to Build

Landing page

Features page

Pricing page

Signup UI

Login UI

Resume upload UI

Dashboard UI with dummy data

Skill gap UI with dummy data

Roadmap UI with dummy data

ATS resume builder UI with dummy data

### Why Build This First

This allows quick review of product flow before heavy backend and AI integration.

## Deliverables

- Clickable frontend prototype
  - Dummy dashboard
  - Dummy roadmap
  - Dummy skill gap result
  - Dummy ATS resume preview
- 

## 8. Phase 6 - Version 1.0 MVP: Resume-First Smart Onboarding

### Goal

Build the first real product flow: user uploads resume first, system auto-fills profile, user edits or completes missing details.

## Screens

/onboarding/upload-resume  
/onboarding/review-profile  
/onboarding/success  
/profile

## Build Sequence

### Step 1: Resume Upload UI

Build:

Drag and drop upload  
Browse file button  
File type validation  
File size validation  
Upload progress  
Error states

Allowed files:

PDF  
DOCX

Max size:

5 MB

### Step 2: Supabase Storage Upload

Flow:

User selects resume  
v  
Frontend validates file  
v  
Upload to Supabase Storage bucket: resumes  
v  
Save metadata in resumes table

### Step 3: Resume Parsing Basic Version

For MVP, use simple parser first:

Email extraction using regex  
Phone extraction using regex  
Skills extraction using skill dictionary  
Links extraction using regex  
Text extraction from PDF/DOCX

AI enhancement can come later.

### Step 4: Auto-Filled Profile Review

Show extracted fields:

Full name  
Email  
Phone  
City  
Education  
Work experience

Skills  
Projects  
LinkedIn  
GitHub  
Portfolio

User manually fills:

Target job role  
Preferred location  
Work preference  
Expected salary  
Notice period  
Career goal

### Step 5: Save Final Profile

Save to:

profiles  
profile\_field\_sources  
resumes

Mark:

onboarding\_completed = true

### Backend APIs

POST /api/onboarding/resume-upload  
POST /api/onboarding/auto-fill  
GET /api/onboarding/profile  
PUT /api/onboarding/profile  
POST /api/onboarding/complete

### Deliverables

- Resume-first onboarding working
  - Resume upload working
  - Basic parsing working
  - Auto-filled profile form working
  - Manual edit and completion working
  - Profile saved to Supabase
  - User redirected to dashboard
- 

## 9. Phase 7 - Version 1.0 MVP: Dashboard

### Goal

Create the main user dashboard with real data.

### Dashboard Cards

Profile Completion  
Resume Match Score  
Target Role  
Skill Gap Summary  
Roadmap Progress

Pending Tasks  
ATS Resume Status  
Reminder Status

## Build Sequence

1. Fetch profile data.
2. Fetch latest resume.
3. Fetch latest skill analysis.
4. Fetch active roadmap.
5. Fetch pending tasks.
6. Display empty states if data not available.

## Empty States

No resume uploaded -> Upload your resume  
No skill gap -> Analyze skill gap  
No roadmap -> Generate roadmap  
No ATS resume -> Build ATS resume

## Deliverables

- Dashboard route working
  - Real Supabase data displayed
  - Empty states ready
  - CTA buttons connected to correct routes
- 

# 10. Phase 8 - Version 1.0 MVP: Skill Gap Analysis

## Goal

Compare user skills with selected target job role and show missing skills.

## Build Sequence

### Step 1: Job Role Selection

Fetch from:

job\_roles

### Step 2: Required Skills Fetch

Fetch from:

role\_skills

skills

### Step 3: Skill Matching Logic

MVP matching logic:

Normalize user skills

Normalize required skills

Exact match

Alias match

Calculate match score

Identify missing skills

## Step 4: Store Analysis

Save to:

```
skill_analyses  
skill_analysis_items
```

## Step 5: Show Result

Display:

```
Match score  
Current skills  
Matched skills  
Missing skills  
Recommended skills  
Learning priority
```

## APIs

```
GET /api/job-roles  
GET /api/job-roles/:roleId/skills  
POST /api/skill-gap/analyze  
GET /api/skill-gap/:analysisId  
GET /api/users/me/skill-gap/latest
```

## Deliverables

- Target role selection working
  - Skill comparison working
  - Match score working
  - Missing skills displayed
  - Result saved in database
  - Dashboard updated with skill gap
- 

# 11. Phase 9 - Version 1.0 MVP: Basic Roadmap

## Goal

Generate a simple roadmap from missing skills before adding full RAG.

## Build Sequence

1. Use missing skills from latest skill analysis.
2. Create basic week-wise roadmap using predefined templates.
3. Save roadmap to database.
4. Save weeks and tasks.
5. Show roadmap on frontend.
6. Allow user to mark tasks complete.

## Tables

```
roadmaps  
roadmap_weeks  
roadmap_tasks
```

## APIs

```
POST /api/roadmap/generate
GET /api/roadmap
GET /api/roadmap/:roadmapId
POST /api/roadmap/tasks/:taskId/complete
POST /api/roadmap/tasks/:taskId/reopen
GET /api/roadmap/:roadmapId/progress
```

## Deliverables

- Basic roadmap generation working
  - Week-wise tasks created
  - Task completion working
  - Progress percentage working
  - Roadmap visible in dashboard
- 

## 12. Phase 10 - Version 1.1: ATS Resume Builder

### Goal

Allow users to generate, edit, preview, and download ATS-friendly resumes.

### Build Sequence

#### Step 1: Resume Builder UI

Build sections:

```
Summary
Skills
Education
Projects
Experience
Certifications
Links
```

#### Step 2: Generate Resume Content

Initial version can use template-based generation.

Later AI version improves:

```
Summary
Project descriptions
ATS keywords
Skill arrangement
```

#### Step 3: Preview Resume

Create clean resume preview layout.

#### Step 4: PDF Download

Use backend PDF generation:

```
Puppeteer
or PDFKit
```

## Step 5: DOCX Download

Use:

docx npm package

## Step 6: Store Generated Files

Store in Supabase Storage:

generated-resumes/{user\_id}/{generated\_resume\_id}.pdf  
generated-resumes/{user\_id}/{generated\_resume\_id}.docx

## Tables

generated\_resumes  
resume\_templates  
usage\_counters

## APIs

POST /api/resume-builder/generate  
GET /api/resume-builder/:resumeId  
PUT /api/resume-builder/:resumeId  
GET /api/resume-builder/:resumeId/preview  
GET /api/resume-builder/:resumeId/download/pdf  
GET /api/resume-builder/:resumeId/download/docx

## Deliverables

- Resume builder UI ready
  - Resume content generated
  - User can edit sections
  - Preview working
  - PDF download working
  - DOCX download working
  - Generated resume saved
- 

# 13. Phase 11 - Version 1.2: AI Service and RAG Roadmap

## Goal

Upgrade roadmap generation using AI, embeddings, and Supabase pgvector.

## AI Service Stack

Python  
FastAPI  
pdfplumber / PyMuPDF  
python-docx  
spaCy / regex  
LangChain or LlamaIndex  
Supabase pgvector  
LLM provider  
Embedding provider

## AI Service Build Sequence

### Step 1: Setup FastAPI Service

Routes:

```
POST /ai/parse-resume
POST /ai/extract-skills
POST /ai/generate-roadmap
POST /ai/generate-resume
POST /ai/embed-document
```

### Step 2: RAG Knowledge Base

Create admin upload system for:

```
Job role requirements
Skill descriptions
Interview topics
Learning resources
Roadmaps
ATS keywords
Resume tips
```

Store in:

```
knowledge_base_documents
```

### Step 3: Generate Embeddings

For each document:

```
content -> embedding -> store in pgvector
```

### Step 4: RAG Retrieval

Flow:

```
User profile + missing skills + target role
v
Create query embedding
v
Search pgvector
v
Retrieve relevant documents
v
Send context to LLM
v
Generate roadmap
```

### Step 5: Save AI Roadmap

Save to:

```
roadmaps
roadmap_weeks
roadmap_tasks
rag_queries
ai_jobs
```



## Deliverables

- FastAPI AI service ready
  - RAG knowledge base ready
  - Embeddings stored in pgvector
  - RAG roadmap generation working
  - AI roadmap saved in database
  - AI job tracking working
- 

## 14. Phase 12 - Version 1.3: Scalability and Security

### Goal

Make the application production-ready.

### Security Tasks

JWT verification  
Supabase RLS  
Admin middleware  
Input validation  
Rate limiting  
File validation  
Private storage buckets  
Signed URLs  
CORS  
Helmet  
Audit logs

### Rate Limiting

Implement with Redis:

Login successful capacity: minimum 100 users/minute system-wide  
Failed login: 5 attempts/minute per IP + email  
OTP: 3 requests/10 minutes  
Resume upload: 10/hour/user  
Roadmap generation: 20/day/user  
ATS resume generation: 10/day/user

### Queue System

Use BullMQ for:

emailQueue  
resumeParsingQueue  
roadmapGenerationQueue  
resumeBuilderQueue  
weeklyReminderQueue

### Workers

Build:

Email worker  
Resume parsing worker  
Roadmap generation worker  
ATS resume worker

## Deliverables

- Redis rate limiting working
  - Queue system working
  - Workers running
  - Security middleware ready
  - Audit logging ready
  - Private storage access working
- 

## 15. Phase 13 - Version 1.4: Progress Tracking and Weekly Reminders

### Goal

Track roadmap progress and send weekly reminder emails if user is not completing tasks.

### Build Sequence

#### Step 1: Progress Tracking

Calculate:

Total tasks  
Completed tasks  
Pending tasks  
Overdue tasks  
Progress percentage

#### Step 2: Reminder Logic

Rules:

If current week task is pending -> send weekly reminder  
If user inactive for 7 days -> send inactive reminder  
If progress is below expected -> send motivational email  
Do not send duplicate reminder for same week

#### Step 3: Scheduler

Create scheduler service:

Runs weekly  
Finds users with pending tasks  
Creates reminder jobs  
Saves reminder logs

#### Step 4: Email Worker

Send:

Weekly pending task email  
Inactive user email  
Motivational email

#### Step 5: UI Updates

Add to dashboard:

Reminder status  
Pending tasks

Last reminder sent  
Roadmap progress

## Tables

reminder\_logs  
notifications  
email\_logs  
roadmap\_tasks

## APIs

POST /api/reminders/check-weekly  
GET /api/reminders/logs  
GET /api/reminders/logs/:userId

## Deliverables

- Roadmap progress tracking working
  - Weekly scheduler working
  - Reminder emails working
  - Reminder logs saved
  - Dashboard reminder status visible
  - Duplicate reminder protection working
- 

# 16. Phase 14 - Payment and Upgrade Flow

## Goal

Add paid plans and usage limits.

## Plans

Free  
Pro  
Premium

## Build Sequence

### Step 1: Plans Table

Create plan limits:

resume\_upload\_limit  
roadmap\_generation\_limit  
ats\_resume\_generation\_limit  
ai\_chat\_limit

### Step 2: Usage Counters

Track monthly usage:

resume\_upload  
roadmap\_generation  
ats\_resume\_generation  
ai\_chat

### Step 3: Billing UI

Build:

Current plan  
Usage limits  
Pricing cards  
Upgrade button  
Billing history

### Step 4: Payment Gateway

Recommended:

Razorpay for India  
Stripe for international

### Step 5: Webhook Handling

Handle:

payment success  
payment failed  
subscription cancelled  
subscription expired

### Tables

plans  
subscriptions  
payment\_transactions  
usage\_counters

### APIs

GET /api/billing/plans  
GET /api/billing/current-plan  
POST /api/billing/create-checkout  
POST /api/billing/webhook  
GET /api/billing/history

### Deliverables

- Pricing page connected
  - Billing page working
  - Usage limits working
  - Upgrade modal working
  - Payment checkout working
  - Subscription status saved
- 

## 17. Phase 15 - Admin Panel

### Goal

Allow admin to manage platform data and monitor system.

## Admin Pages

/admin  
/admin/users  
/admin/job-roles  
/admin/knowledge-base  
/admin/reminders  
/admin/logs

## Admin Features

View users  
View user onboarding status  
Manage job roles  
Manage required skills  
Upload knowledge base documents  
Generate embeddings  
View reminder logs  
View failed AI jobs  
View audit logs

## APIs

GET /api/admin/users  
GET /api/admin/users/:userId  
POST /api/admin/job-roles  
PUT /api/admin/job-roles/:roleId  
DELETE /api/admin/job-roles/:roleId  
POST /api/admin/knowledge-base  
GET /api/admin/knowledge-base  
DELETE /api/admin/knowledge-base/:documentId  
GET /api/admin/analytics  
GET /api/admin/logs

## Deliverables

- Admin dashboard ready
  - Job role management ready
  - Skill management ready
  - Knowledge base management ready
  - Reminder logs visible
  - Failed jobs visible
- 

## 18. Phase 16 - Testing

### Goal

Make sure the complete system works reliably.

### Frontend Testing

Test:

Signup form  
Login form  
Resume upload UI  
Profile review form

Dashboard cards  
Skill gap page  
Roadmap task completion  
Resume builder  
Billing flow

## Backend Testing

Test:

Auth middleware  
Rate limits  
Resume upload  
Profile update  
Skill gap logic  
Roadmap generation  
Task completion  
Reminder scheduler  
Payment webhook  
Admin APIs

## AI Testing

Test:

Resume text extraction  
Skill extraction  
Roadmap JSON format  
ATS resume generation  
RAG retrieval quality  
AI timeout handling

## Integration Testing

Full flows:

Signup -> Verify -> Upload Resume -> Auto-fill -> Dashboard  
Profile -> Skill Gap -> Roadmap -> Complete Tasks  
Roadmap Pending -> Reminder Email  
Profile -> ATS Resume -> PDF Download  
Free Limit -> Upgrade -> Payment Success

## Deliverables

- Unit tests added
  - API tests added
  - Integration tests completed
  - AI output validation done
  - Critical bugs fixed
- 

# 19. Phase 17 - Deployment

## Goal

Deploy full project to production.

## Recommended Deployment

Frontend: Vercel / Netlify  
Backend: Docker on VPS / Render / Railway / AWS EC2  
AI Service: Docker on VPS / Render / AWS EC2  
Database: Supabase Cloud  
Storage: Supabase Storage  
Redis: Upstash Redis / Redis Cloud  
Nginx: Reverse proxy  
CI/CD: GitHub Actions

## Deployment Steps

### Step 1: Environment Setup

Create production environment variables:

SUPABASE\_URL  
SUPABASE\_ANON\_KEY  
SUPABASE\_SERVICE\_ROLE\_KEY  
REDIS\_URL  
AI\_SERVICE\_URL  
EMAIL\_PROVIDER\_API\_KEY  
JWT\_SECRET  
FRONTEND\_URL  
PAYMENT\_SECRET

### Step 2: Build Docker Images

backend image  
ai-service image  
worker image  
scheduler image

### Step 3: Setup Nginx

Routes:

/api -> backend  
/ai -> ai-service

### Step 4: Deploy Workers

Deploy:

email worker  
resume parsing worker  
roadmap worker  
reminder worker

### Step 5: Run Health Checks

Check:

/api/health  
/api/health/db  
/api/health/redis  
/api/health/ai-service

## Step 6: Setup Monitoring

Use:

Sentry

UptimeRobot

Server logs

Supabase logs

Redis logs

## Deliverables

- Frontend deployed
  - Backend deployed
  - AI service deployed
  - Workers deployed
  - Scheduler deployed
  - Supabase connected
  - Redis connected
  - Domain connected
  - HTTPS enabled
  - Health checks passing
- 

## 20. Phase 18 - Final Polish

### Goal

Make the product feel professional and production-ready.

### UI Polish

Improve spacing

Fix mobile responsiveness

Add skeleton loaders

Add empty states

Add error states

Add success toasts

Improve modals

Improve dashboard visuals

Improve roadmap cards

Improve resume preview

### UX Polish

Clear next-step CTAs

Better onboarding messages

Helpful AI explanation

Better error recovery

Upgrade prompts

Reminder banners

### Performance Polish

Cache dashboard summary

Optimize database queries

Compress images

Lazy load routes



Reduce bundle size  
Queue slow AI tasks

## Security Polish

Review RLS policies  
Review storage policies  
Check exposed env variables  
Check API permissions  
Check admin routes  
Test rate limits

## Deliverables

- Final UI review completed
  - Mobile layout fixed
  - Loading states added
  - Error handling polished
  - Security review completed
  - Performance review completed
  - Production release ready
- 

# 21. Version-Based Release Plan

## Version 0.1 - Prototype

### Build

Landing page  
Auth UI  
Onboarding UI  
Dummy dashboard  
Dummy skill gap  
Dummy roadmap  
Dummy resume builder

### Deliverable

Clickable demo.

---

## Version 1.0 - MVP

### Build

Supabase Auth  
Resume-first onboarding  
Resume upload  
Profile auto-fill  
Manual profile edit  
Skill gap analysis  
Basic roadmap  
Dashboard  
Docker local setup

## **Deliverable**

First usable product.

---

## **Version 1.1 - ATS Resume Builder**

### **Build**

AI resume content  
Resume editor  
Resume preview  
PDF download  
DOCX download  
ATS keywords

### **Deliverable**

Users can generate job-ready resumes.

---

## **Version 1.2 - RAG and GenAI Roadmap**

### **Build**

AI service  
Knowledge base  
Embeddings  
Supabase pgvector  
RAG retrieval  
GenAI roadmap  
Career assistant basics

### **Deliverable**

Personalized intelligent roadmap.

---

## **Version 1.3 - Scalability and Security**

### **Build**

Redis rate limits  
Queues  
Workers  
Private storage  
Audit logs  
Nginx  
Production security

### **Deliverable**

Production-ready backend foundation.

---

# Version 1.4 - Progress Tracking and Reminder System

## Build

Task progress  
Weekly reminders  
Inactive user reminders  
Email worker  
Reminder logs  
Dashboard reminder status

## Deliverable

User retention and consistency system.

---

# Version 2.0 - Advanced Platform

## Build Later

Recruiter portal  
College dashboard  
AI mock interview  
Job matching  
Mobile app  
Payment plans advanced  
Multi-resume support

---

## 22. Team Execution Plan

### Suggested Sprint Breakdown

| Sprint    | Focus                         | Duration  |
|-----------|-------------------------------|-----------|
| Sprint 0  | Setup + planning              | 2-3 days  |
| Sprint 1  | Supabase + backend base       | 1 week    |
| Sprint 2  | Frontend base + auth          | 1 week    |
| Sprint 3  | Resume onboarding             | 1 week    |
| Sprint 4  | Dashboard + skill gap         | 1 week    |
| Sprint 5  | Basic roadmap                 | 1 week    |
| Sprint 6  | ATS resume builder            | 1-2 weeks |
| Sprint 7  | RAG AI service                | 2 weeks   |
| Sprint 8  | Redis, queues, reminders      | 1-2 weeks |
| Sprint 9  | Payments + admin              | 1-2 weeks |
| Sprint 10 | Testing + deployment + polish | 1-2 weeks |

---

## 23. Final Implementation Checklist

### Foundation

- [ ] Repo created
- [ ] Frontend setup
- [ ] Backend setup
- [ ] AI service setup

- ☐ Docker setup
- ☐ Supabase setup
- ☐ Redis setup

## Core Product

- ☐ Signup/login
- ☐ Email verification
- ☐ Resume upload
- ☐ Resume parsing
- ☐ Profile auto-fill
- ☐ Manual profile editing
- ☐ Dashboard
- ☐ Skill gap analysis
- ☐ Basic roadmap

## AI Features

- ☐ AI resume parser
- ☐ Skill extractor
- ☐ RAG knowledge base
- ☐ Embeddings
- ☐ RAG roadmap
- ☐ ATS resume generator

## Retention Features

- ☐ Roadmap task tracking
- ☐ Weekly reminder scheduler
- ☐ Reminder email worker
- ☐ Reminder logs

## Monetization

- ☐ Plans
- ☐ Usage limits
- ☐ Upgrade modal
- ☐ Payment gateway
- ☐ Billing history

## Production

- ☐ RLS policies
- ☐ Rate limiting
- ☐ Logging
- ☐ Monitoring
- ☐ Docker deployment
- ☐ HTTPS
- ☐ Final testing

---

## 24. Final Recommendation

Start with **Version 1.0 MVP** only:

Auth

Resume-first onboarding

Resume upload  
Profile auto-fill  
Skill gap  
Basic roadmap  
Dashboard

After MVP works properly, add:

ATS Resume Builder  
RAG Roadmap  
Weekly Reminders  
Payments  
Admin Panel

This approach prevents overbuilding and helps complete the project step by step like a real product team.